

Lec 23c-

Preprocessor:-

Preprocessor directives are called header file

#include <iostream.h>

using namespace std;

currently .h is removed

h → extension / header file

Include directive:-

#include after given input show output

→ We connect any other file in header file

→ file created by yourself don't use "<>".

#include "my file.h".

Define directive:-

→ define directive → macro

→ Macro → special name

→ #define name → special macro name
value is exactly define

PI = 3.14 → variable

#define PI 3.141652 → Macros.

→ if we use ; it is syntax error
 → program Pg 286.

```
#include <iostream>
using namespace std;

#define PI 3.1415926
{
  int radius = 5;
  cout << "Area of circle with radius " << radius << " = " << PI * r * r;
}
```

→ Computer will take accurate value of π .

Preprocessor directives:-

#include <file name>

#include "filename.h"

#define

#if def

#undef

#endif

#if def

#if end of

#if

#else

#end if

#error

#line

#pragma

#assert

Preprocessor directives start with "#".

#ifdef PI

..... then do something

#endif

Def $\rightarrow$ its value will be fixed it will not change.

undef $\rightarrow$ for changing the constant value fixed in #def we used #undef

Program $\rightarrow$ 288.

CONIO.h $\rightarrow$ getch() undeclared.
Just used as console input output

Macros:

There are two categories of Macro.

1. Using #define PI
2. Macros Parameterized with Arguments

#define square(x) $x * x$

#define ^{square} (i+j) :

$i + j * i + j$

Use bracket with Program

* Program $\rightarrow$ 291

brackets must used with Macro

Lec 24c-

Dynamic Memory Allocation:-

→ we allow memory to store data in itself

- {

Static

Dynamic

→ make variable

`int a=10, int arr[10], char s[20];`

this is also called compile time allocation
- When we know variable length, we use Static
- Static Memory stores in "Stack"
- When we don't know length of variable.
- Dynamic used and stored as "Heap".
- Heap will change its size
- The value will assigned, when the Program executes.

★ How create void pointer?

```
int* i;
```

```
char* s;
```

For create void pointer, we use casting.

for every pointer, we use data type

```
void* ptr;
```

cast it before use.

★ One datatype to other datatype
conversion

- void * ptr
- Name the datatype use
- write in parenthesis and then pointer

(int *) ptr;

- Pointer → Null value
must use header file (stdlib.h, stddef).
one of above
- Null Pointer → Special type of pointer
with all zero values
- All zero is an invalid Memory
Address

★ Calloc function:-

- Makes multiple blocks of memory

★ Real Loc Function:-

- function which is used to resize the memory block
created by alloc & malloc

★ Malloc function:-

- Makes only one block of memory

* Memory Leakage:-

- it occurs when Programmers allocates memory by using new ~~for~~ keyword and forgets to deallocate the Memory by using delete () function.
- it occurs by using wrong delete operator.
- Don't free up the memory.
- not useable memory must deallocate
- When memory is allocated, but no deallocated.

* Dangling Pointers:-

- Dangling pointers is a situation where you have valid pointers in the stack, but it is pointing to invalid memory.
- Pointer that occurs at the time when the object is de-allocated from memory without modifying the value of pointer.

* Free ();

When we don't want extra memory, we delete that memory, then it will be free ().

Lec 25.

M T W T F S

* INLINE FUNCTION:- ^{Part 2}

```
#include <iostream>
using namespace std;

inline void display(int a)
{
    cout << a << endl;
}

main()
{
    display(10);
}
```

- A simple function take more time for execution
- But if we used inline keyword, the execution will be faster.

Situations:-

- Inline works if function are small.
- if a value is return in function, that contain static variable, or goto statement, then no inline function is used.

Advantages → function is fast.

Date: _____

Program Explanation. → 316

→ Macros will mostly give ~~answe~~ wrong / and ~~two~~ time more than actual.

* FUNCTION OVERLOADING:-

→ (<<) stream insertion operator

→ iostream.h contains prototype for all those functions.

cout << "This is my string";

→ function will be overload when same function name work with different arguments

void print (int i)

" print (double)

" " (~~char~~ "c")

program — overview

Part 15

Structured Programming:-

→ When we want to create long program, we break it into small modules or pieces corresponds to a function. This process is called top-down structured programming.

- Divide & Conquer.
- Single Entry single Exit.
- Comment your Programs

LIMITATIONS OF STRUCTURED PROGRAMMING.

→ Handouts → Read.

→ Default function Arguments.

```
#include <iostream>
```

```
using namespace std;
```

```
void run (int x, double y)
```

```
{
```

```
    cout << x << endl;
```

```
    cout << y << endl;
```

```
}
```

```
main ()
```

```
{
```

```
    run(30, 10.5);
```

```
}
```

```
void run (int x=3,
```

```
    y=3.1)
```

↓

default
argument

```
run(30)
```

→ When we call function and give another value, by default argument is removed.

Lec 27c-

- Theory → Once read from handout
- Program → folder CS201P Lec 27
- Construction
- Destructor.

Lec 28c-

→ Memory Allocation in C:-

- directly object created using `(.)` dot operator
- if work with pointer, the member will access through `[>]` operator
- `malloc()`, `alloc()`, `Realloc()` create space in memory.
- if your program is running, don't worry about memory. Memory allocated at runtime is called **Heap**.
Memory available at runtime is called **Free store**.

→ Memory Allocation in C++:-

new int;

→ if you use int data type with word new

int *iptr

iptr = new int;

→ For creating memory we use new and for (delete) we use to free memory

* New Operator & Classes:-

- Whenever New operator is used in classes
- All variables increase their size
- When objection is created, variable create memory.
- pg e 351.
- new is used accessed with (>).

* Classes and Structures in C++

- classes used class keyword. → By default private.
- Structure used struct keyword
↓
By default Public
visibility
- write once Public or Private.

Program → folder

* Friend Classes -

A friend class is a class that can access the private and protected members of a class in which it is declared as friend.

Program → folder

Lec 30:-

Reference Data type in C++:-

→ A reference is an alternate name to an existing variable

```
int i;  
int &j = i;  
i, j output are same
```

Dangling Reference in C++:-

→ It is a dangling reference is reference of an object that has gone out of scope or otherwise destroyed.

→ the variable is removed

Lec 31 → Handouts
Lec 32 → Handouts

Lec 33c-

★ Use of this pointer in C++.

$k = k;$

→ This pointer is pointer which stores memory address for other variable

Program → folder (C++01) practice

Lec 34c-

Array of Code:-

→ The Array of object represent storing multiple object in a single name.

→ data is accessed randomly by using index number

→ Reduce The time and memory by storing data in single variable

→ New is called before constructor

→ Delete is called after destructor

New operator & Constructor:-

- new operator is called to create an object
- New operator is used inside constructor.
- For different data member, use dynamic memory allocation
- Responsibility to deallocate memory

delete operator & classes:-

- It is just used to delete the taken memory.
- It is used to free up the memory that is allocated.

Pg 359-361.

Lec 29:-

Friend function:-

- A friend function is a function that is specified outside a class but has the ability to access the class members protected & private data.

Date: _____

M T W T F S

- A global array is called **Pool** that can store all the name objects expected.

Lec 35-

- A C++ class is a collection of data and the method necessary to control and maintain that idea.
- A class is a data type, analogue to ints, floats and double.
- A C++ object is a specific variable having a class as its data type.
- cin cout are special Pre defined objects with different classes.
- A C++ stream is a flow of data into or out of a Program.
- We convert it into four classes.
- **istream** → istream
 - **ostream** → out stream
 - **ifstream** → input file stream.
 - **ofstream** → output file stream.

Lec 35-

Manipulators-

→ **Non Parameterized** Manipulators
do not take control formatting of output as parameter argument to input or

Def:-

- Manipulators are special functions that can be include in I/O statement to alter the format parameter of stream.
- Manipulators are operators that are used to format the data display
- To access manipulator, the file `<manip>` should be include in Program.

Types:-

- Parameterized Manipulator.
- Non-Parameterized Manipulator

Lec 37:-

- In C++, stream insertion "<<" is used for output and extraction ">>" for input
- `cout` is object of `ostream`
- `cin` is object of `istream`
- operators are overloaded as global function

→ **Non Parameterized** Manipulators
do not take control formatting of output as parameter argument to input or

Def:-

- Manipulators are special functions that can be include in I/O statement to alter the format Parameter of stream.
- Manipulators are operators that are used to format the data display.
- To access manipulator, the file `<manip>` should be include in Program.

Types:-

- **Parameterized Manipulator.**
- **Non-Parameterized Manipulator**

Lec 37:-

- In C++, stream insertion "<<" is used for output and extraction ">>" for input.
- `cout` is object of `ostream`.
- `cin` is object of `istream`.
- operators are overloaded as global function.

→ **Non Parameterized** Manipulators
do not take argument to control formatting of input or output as parameter

Def:-

- Manipulators are special functions that can be include in I/O statement to alter the format parameter of stream.
- Manipulators are operators that are used to format the data display.
- To access manipulator, the file to manip should be include in Program.

Types:-

- Parameterized Manipulator.
- Non-Parameterized Manipulator

Lec 37:-

- In C++, stream insertion "<<" is used for output and extraction ">>" for input.
- cout is object of ostream
 - cin is object of istream.
 - operators are overloaded as global function

→ **Non Parameterized** Manipulators
do not take control formatting of argument to output as parameter input or

Def:-

- Manipulators are special functions that can be include in I/O statement to alter the format parameter of stream.
- Manipulators are operators that are used to format the data display.
- To access manipulator, the file `<iomanip>` should be include in Program.

Types:-

- Parameterized Manipulator.
- Non-Parameterized Manipulator

Lec 37:-

- In C++, stream insertion "`<<`" is used for output and extraction "`>>`" for input.
- `cout` is object of `ostream`.
- `cin` is object of `istream`.
- operators are overloaded as global function.

→ Stream (<<) and extraction (>>) are always implemented as non-member function.

Lec 38.-

User defined Manipulator:-

C++ Provides many Predefined manipulators by you can also create your own manipulators

Syntax:-

```
ostream& manipulator_name (ostream& ostr)
{
    set of statements;
    return ostr;
}
```

Lec 39.-

Study with Khalid, Alirza
facebook
Hardouts

Date: _____

MTWTFSS

Lec 40-41

→ Program → classes inside class

Handouts → Reading
Program Practice

Lec 41-

Templates:-

- Templates in C++ is defined as blueprint or formula for creating a generic class or function.
- C++ is generic function template.

Lec 42 - Lec 44 → Read. Handout

CS201

→ must know about classes and objects

→ Program:-

```
#include <iostream>
using namespace std;
```

```
class Person {
    Private:
```

```
    Public:
```

```
    void talk()
```

```
    {
```

```
        cout << "A Person can talk" << endl;
```

```
    }
```

```
    void walk()
```

```
    {
```

```
        cout << "A person can walk" << endl;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Person P; //object creation.
```

```
    P.talk(); //function called
```

```
    P.walk();
```

→ Constructor working within class.

Program:-

```
#include <iostream>
using namespace std;

class construct
{
    public:
        int a;
        int b;
        construct () {                // constructor
            a = 10;
            b = 30;
        }
};

main () {
    construct c;                      // object creation
    cout << "a: " << c.a << endl;
    cout << "b: " << c.b;
}
```

A constructor in C++ is a special "Member function" having same name as that of class which is used to initialize some values to the data members of object.

Lec 26:-

Classes & Objects:-

- Practical in cs201 folder

```
#include <iostream>
```

```
using namespace std;
```

```
Class student
```

```
{
```

```
// Access modifier
```

```
// public, private, protected
```

work with

→ variable

→ function,

```
}
```

```
main()
```

- (.) is used to call object
- if we already create a object of a class, then we will easily access the data of class in any place.
- work of object is to give the permission to get inside class

Constructor:-

- its name is same as name of class.
- it is called by object
- CS201 → folder → example

Date: _____

M T W T F S

→ #include < ~~using~~ iostream >
using namespace std;

```
class Teacher  
{
```

Public:

```
Teacherint (i)  
{  
    cout << "I am teacher";  
    3 = 100  
    4 = 279  
}
```

```
};
```

```
int main ()
```

```
{
```

```
    Teacher T;
```

```
    T = Teacher();
```

```
    cout << t.x << endl;
```

```
    cout << t.y << endl;
```

```
}
```

★ Important topics of final
terms -

- Variables
- how to Take output in C++.
- if/else
- loops.

- functions.
- pointer.
- Arrays.

→ Program for create variable.

```
#include <iostream>
using namespace std;
main ()
{
    // Declaration or create
    int x, y;
    int age;
    string name;
    // Definition.
    age = 99;
    name = "Malailca";
    Cgpa = 3.5;
```

→ how to take output.

```
cout << " " << endl;
cout << "here we go" << endl;
```

→ Use of if/else.

```
main ()
{
    if ()
        Statement;
    else;
```

```
main ( )  
{  
    if (33 == 33)  
    {  
        cout << "Yes" << endl;  
    }  
    else  
    {  
        cout << "No" << endl;  
    }  
}
```

- While loop ()
- do while loop ()
- for loop ()

* for:-

- Starting Point
- Ending Point
- Incr / decr

* While loop:-

```
Starting ptr ← int s = 1;  
while (s <= 5) → ending point  
{  
    cout << s << endl;  
    s++; → Incr / decr  
}
```

CS201
TIPS.

→ MCQs Files.

→ 23-30 → lectures.

→ 36-39 → lectures.

→ Handwritten notes.

→ MCQ files.

→ Current Papers.

→ Subjective files.

Placement of Variable Declaration

```
#include <iostream>
```

```
using namespace std;
```

```
int x = 100; → global variable.
```

```
main () {
```

```
    int x; → declaration
```

```
    x = 10; → definition.
```

```
    int x = 10; → local variable.
```

```
    cout << x;
```

```
}
```

Answer is 10.

→ ~~global~~ **local** variable is used inside braces

→ **global** variable is used outside braces.

```
main ()
```

```
{
```

```
    int x;
```

```
    x = 10;
```

```
    cout << x;
```

```
}
```

→ global var called.